This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

MalPacDetector: An LLM-based Malicious NPM Package Detector

Jian Wang, Zhen Li, *Member, IEEE,* Jixiang Qu, Deqing Zou, Shouhuai Xu, *Senior Member, IEEE,* Ziteng Xu, Zhenwei Wang, Hai Jin, *Fellow, IEEE*

***Abstract*—The Node Package Manager (NPM) registry contains millions of JavaScript packages widely shared between worldwide developers. However, NPM has also been abused by attackers to spread malicious packages, highlighting the importance of detecting malicious NPM packages. Existing malicious NPM package detectors suffer from, among other things, high false positives and/or high false negatives. In this paper, we propose a novel Malicious NPM Package Detector (MalPacDetector), which leverages Large Language Model (LLM) to automatically and dynamically generate features (rather than asking experts to manually define them). To evaluate the effectiveness of MalPacDetector and existing detectors, we construct a new NPM package dataset, which overcomes the weaknesses of existing datasets (e.g., a small number of examples and a high repetition rate of malicious fragments). The experimental results show that MalPacDetector outperforms existing detectors by achieving a false positive rate of 1. 3% and a false negative rate of 7. 5%. In particular, MalPacDetector detects 39 previously unknown malicious packages, which are confirmed by the NPM security team.**

***Index Terms*—Malicious package detection, npm, malicious features, large language model.**

I. Introduction

JAVASCRIPT is widely used in web and mobile apps. To ease the sharing and distribution of JavaScript code and its dynamic maintenance, the Node Package Manager (NPM) is used to automate the installation, configuration, and upgrade of JavaScript packages deposited in the NPM registry, which is a database of JavaScript packages (including both the code and the associated metadata). As a result, the NPM is the default package manager for the `Node.js` environment. The NPM hosts more than 2 million reusable packages [1]. This popularity has made it a target of attackers for spreading malicious JavaScript packages. [2]–[4]. For instance, the *eslint-scope* attacker first compromises the NPM account of the maintainer of the `ESLint` package and then publishes malicious versions of the `eslint-scope` and `eslint-config-eslint` packages, which will download and execute the malicious payload on a victim computer running these malicious packages to steal the sensitive data stored in the victim computer's `.npmrc` file (typically the victim user's credentials for publishing packages at NPM) [5]. In another attack, the `getcookies` package contains a backdoor that enables an attacker to execute arbitrary code at a victim computer [6].

Attacks against NPM, including those mentioned above, have motivated the investigation of malicious NPM package detectors. Existing detectors follow two approaches: *program analysis* vs. *machine learning*. The program analysis approach detects malicious packages mainly via pattern matching, clone detection, and dynamic analysis [7]–[10]. However, this approach often uses rules to detect malicious packages and result in high false positives because these rules are relatively general [8]. Moreover, some tools are heavyweight or not practical [7]. The machine learning approach uses expert-defined features to train detectors [11]–[13]. The effectiveness of this approach depends on the quality of the features. However, high-quality features are often tedious to define, difficult to define (e.g., when the attack exploits obfuscation techniques), and subjective to define (i.e., different experts may define different features). These issues explain why existing detectors are ineffective (e.g., incurring high false positives and/or high false negatives [8], [12]) and difficult to use (i.e., heavyweight or update rules manually [7]), and have limited use cases [14].

Our contributions. In this paper, we make three contributions. First, we propose Malicious NPM Package Detector (MalPacDetector), which makes Large Language Model (LLM) able to pay attention to, and thus automatically identify NPM malicious code fragments and generate features (i.e., we do not need human experts to define features). Moreover, MalPacDetector can automatically and dynamically update the features as malicious behaviors evolve.

Second, we propose a set of desired characteristics of benchmark datasets to evaluate malicious NPM package detectors. These characteristics guide us in collecting a new benchmark

Manuscript received September 28, 2024. The authors from Huazhong University of Science and Technology were supported by the National Natural Science Foundation of China under Grant No. 62272187. Shouhuai Xu's research was not supported by any funding agency. *(Corresponding author: Zhen Li.)*

Jian Wang, Zhen Li, Jixiang Qu, and Deqing Zou are with the National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Laboratory, Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, JinYinHu Laboratory, Wuhan, China (e-mail: hust_jianw@hust.edu.cn; zh_li@hust.edu.cn; qujixiang@hust.edu.cn; deqingzou@hust.edu.cn).

Shouhuai Xu is with the University of Colorado Colorado Springs, Colorado Springs, USA (e-mail: sxu@uccs.edu).

Ziteng Xu and Zhenwei Wang are with Ant Technology Group Co., Ltd, Hangzhou, China (e-mail: ziteng.xzt@antgroup.com; zhenwei.wzw@antgroup.com).

Hai Jin is with the National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Laboratory, Cluster and Grid Computing Laboratory, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China (e-mail: hjin@hust.edu.cn).

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

dataset, dubbed MalnpmDB. When compared with existing datasets, MalnpmDB has the following advantages: It contains 3,258 malicious packages, including 7 types of attacks and 4,051 benign packages; by contrast, existing datasets focus on a single type of attack and have a high repetition rate of malicious code fragments [7], [14], [15].

Third, we use MalnpmDB to evaluate the effectiveness of MalPacDetector and the existing detectors that are publicly available. Experimental results show MalPacDetector outperforms the existing detectors by achieving a 98.0% precision, a 1.3% false positive rate, a 7.5% false negative rate, and a 95.2% F1-measure; this represents a 2.9% higher precision and a 3.8% higher F1-measure when compared with the state-of-the-art machine learning-based detector [11]. In particular, MalPacDetector detects 39 previously unknown malicious packages, which are confirmed by the NPM security team.

To enable others to verify or leverage results, we have made the source code of MalPacDetector and the MalnpmDB dataset publicly available at https://github.com/MalPacDetector/MalPacDetector-core.

Ethical consideration. We reported the 39 previously unknown malicious packages to the NPM security team, which then removed these 39 packages from the NPM registry. Communication emails with the NPM security team are available for verification if necessary.

Paper organization. Section II describes the MalPacDetector. Section III presents the MalnpmDB dataset. Section IV reports our experiments. Section V reviews the related prior studies. Section VI discusses limitations of this study. Section VII concludes the paper.

II. The MalPacDetector

As highlighted in Figure 1, MalPacDetector has two phases: training and detection. In the training phase (Steps 1-3), the input is NPM packages and the output is a trained model; in the detection phase (Steps 4-5), the input is target NPM packages and the trained model, and the output is malicious lines of code in the target NPM packages.

- **Step 1: LLM-based feature generation**. This step uses an LLM to generate features of NPM packages.
- **Step 2: Feature value extraction for training packages**. This step extracts feature values from the training packages.
- **Step 3: Model training.** This step trains a model as a malicious package detector.
- **Step 4: Feature value extraction for target packages**. This step extracts feature values from the target packages.
- **Step 5: Malicious code detection in target packages.** This step detects malicious lines of code in target packages.

These steps are elaborated below.

A. Step 1: LLM-based Feature Generation

Unlike existing malicious NPM package detectors that rely on experts to manually define features, we propose using LLM to automatically generate features as follows.

Step 1.1: Identifying malicious code in training NPM packages. We propose using an LLM to analyze program code in malicious NPM packages (the input), extract malicious code snippets, and summarize their malicious behaviors. The idea is to guide an LLM to focus on malicious code snippets, analyze malicious behaviors, and summarize features. This step applies to malicious, but not benign packages, because we hope to use LLM to learn the malicious behaviors, and thus features, of malicious packages that would not be exhibited by benign packages. This step is conducted iteratively as one needs to query the LLM in question in multiple rounds with increasingly refined (i.e., adaptive) prompts.

As illustrated in Figure 2, this step starts with an initial prompt to ask an LLM to analyze the program files in a malicious NPM package. By standardizing the output format of the LLM, we can automatically process its response to extract malicious code and keywords. These, along with their locations (i.e., the lines of code) and descriptions of the malicious behaviors, are then incorporated into a new prompt to the LLM as comments. The basic idea guiding the design of prompts is to instruct the LLM to summarize malicious behaviors of an input NPM package, so that one can use a keyword extractor (e.g., YAKE! [16]) to characterize the malicious behaviors. We repeat this process until the response from the LLM converges and then use the converged response as the malicious behavior.

Figure 3 presents a running example. Figure 3(a) shows a piece of malicious JavaScript code, which calls a function to retrieve local information, packs the information into a JSON file, and sends it to a remote server controlled by the attacker. Figure 3(b) shows the analysis result by an LLM (i.e., GPT 3.5 [17]), namely the lines of code and their associated malicious behaviors (i.e., packaging users' data and sending the package to a server).

Step 1.2: Building a malicious snippet set. This step is to automatically extract malicious code snippets and malicious behaviors from a malicious package, and construct a malicious snippet set with data structure (malicious snippet, malicious behavior), which is made possible by the way the the prompts are constructed in Step 1.1. Given results in the format shown in Figure 3(b), by limiting the output of LLM (i.e., highlighting malicious behavior), one can automatically extract keywords to reflect malicious behaviors exhibited by the example (e.g., keywords "data leakage" and "remote code execution" in the example shown). Figure 3(c) shows how the malicious snippet set is built based on the output of the LLM as shown in Figure 3(b). The 'code' column is obtained by the LLM identifying malicious code, and its corresponding 'behavior' is obtained by the extractor YAKE! summarizing the output of the LLM.

Step 1.3: Summarizing malicious behavior features. This step is to process the extracted malicious snippet set obtained in Step 1.2, by automatically merging similar malicious behaviors (e.g., "process execution", "child process", and "create process") with the LLM and summarizing their behavior features. Figure 3(d) shows the summarized features for the running example. In this example, most of the malicious code snippets exhibiting the malicious behavior of "data leakage" contain OS-related APIs, meaning the use of operating systems

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

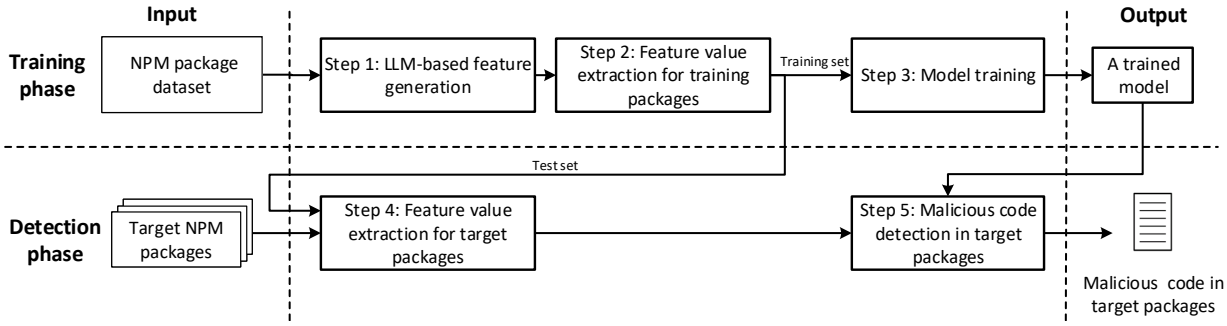

Figure 1: Overview of the training and detection process of MalPacDetector: the training phase generates a trained model and the detection phase uses the trained model to detect malicious lines of code in target NPM packages.

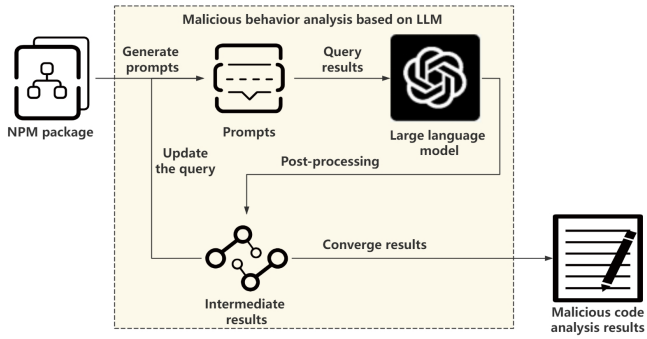

Figure 2: Illustration of the iterative process of using LLM to learn malicious NPM package behaviors

is an important feature, leading to the feature summary of "operating system operation". The code snippets exhibiting the behavior of "remote code execution" generally contain strings such as IP address, so whether IP addresses are involved is a feature that needs to be considered, leading to the feature summary of "HTTP request".

Finally, a feature set, namely the output as shown in Figure 3(e), is generated from the feature summary based on the frequency of malicious behaviors exhibited in the snippet set.

For this purpose, we propose Algorithm 1 to generate features from malicious code fragments. The basic idea is to measure the frequency of malicious behaviors exhibited by malicious code fragments with respect to a given threshold (Line 12). In this way, we can avoid the problem of model hallucination, and a few wrong answers will not affect the results of the feature set. Drawing from the set of malicious code fragments (Line 7), we identify and summarize feature sets by LLM (Line 10), and the frequent behaviors are summarized as features. When the frequency of new malicious behavior surpasses the threshold, we promptly update the feature set to ensure the timeliness of our detector.

B. Step 2: Feature Value Extraction

The goal of this step is to build a feature value extractor based on the feature set obtained in Step 1 to convert NPM packages into feature vectors, which are then used for model training. For this purpose, we parse the NPM package (for example, by a JavaScript compiler *babel* [18]) and convert the file into Abstract Syntax Tree (AST). When compared

Algorithm 1 LLM-based feature generation

Input: Malicious packages dataset D, threshold for feature selection θ
Output: Feature set F

1: Malicious snippet set $S \leftarrow \emptyset$;
2: Feature set $F \leftarrow \emptyset$;
3: **for** each package $d_i \in D$ **do**
4: Ask LLM for malicious code C_i, behavior analysis A_i of d_i;
5: **for** each $c_{ij} \in C_i, a_{ij} \in A_i$ **do**
6: Obtain keywords k_{ij} of a_{ij} to mark behaviors;
7: $S \leftarrow S \cup \{< c_{ij}, k_{ij} >\}$;
8: **end for**
9: **end for**
10: Ask LLM to summarize similar behaviors k_{ij} in S into a feature f_{ij};
11: **for** each feature f_{ij} in S **do**
12: **if** the frequency of f_{ij} is greater than θ **then**
13: $F \leftarrow F \cup \{f_{ij}\}$;
14: **end if**
15: **end for**
16: **return** F;

with direct regex matching, AST preserves syntactic structure information, enabling a better treatment of code minification. Then, we traverse the nodes and match them with the features in our malicious feature set. For JavaScript files, we focus on accessing files and network systems, executing sensitive functions, creating new processes, etc. For the *package.json* file, we focus on the customized script instructions in the file. Finally, we convert the matching results into vectors for machine learning training and detection purposes.

When constructing our feature extractor for machine learning applications, we need to establish a comprehensive mapping between features and their constituent code elements (e.g., sensitive characters, operations, and API calls). Departing from the manual approach of leveraging domain experts, we employ an automated data-driven selection process whereby code elements are chosen according to their occurrence frequency in malicious code snippets. Specifically, we employ AST to structurally analyze the code segments in the malicious snippet set. Each AST node represents a potential code element candidate. We statistically analyze the frequency

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

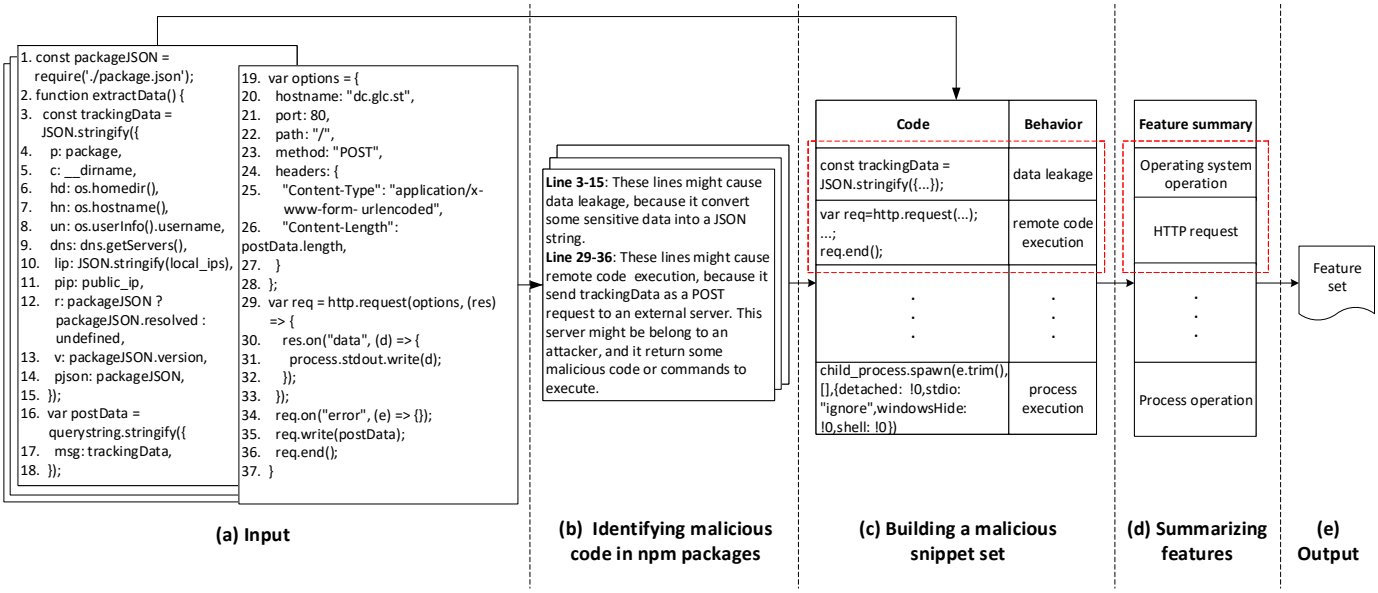

Figure 3: An example of NPM package and the LLM-generated features, where red boxes highlight the resulting code, behavior, and feature summary of this package.

of these code elements. During the manual development of our extraction code, for each malicious behavior pattern, we combine the feature names generated by the LLM with frequency analysis to select the most appropriate code elements as detection targets. These statistical results provide selection scope and priority guidance for our code element selection. This data-driven approach more effectively captures the common characteristics of malicious behaviors. For example, for the malicious behavior of remote code execution in a malicious snippet set, we identify 312 snippets that contain the *eval* function. Consequently, we use *eval* as the detection targets to extract the feature vector of *useEval*. Similarly, for the *useNetwork* features, we choose detection targets such as *http*, *request*, and *node-fetch* that appear frequently in the malicious snippet set.

C. Step 3: Model Training

This is a standard process for training a classifier based on the feature representation of NPM packages, including both malicious and benign ones, to detect malicious packages. The decision on training which classifier can be made based on factors such as the size of the dataset (i.e., the number of examples). When the dataset is small, deep learning models may not be appropriate.

D. Step 4: Target Package Feature Value Extraction

In the detection phase, we first generate the AST of the target package, then extract feature vectors from the nodes of the AST based on our feature set, and finally, detect the malicious target packages using the trained model. While outputting the results, we can also obtain the malicious lines of code for malicious packages.

Figure 4(a) shows a target package, which contains two files: *package.json* and *index.js*. Note that *index.js* imports *exec* from *child_process* and then executes a shell command to collect sensitive information and sends it to an outside server. Recall that Step 1 leads to features related to the installation and execution of scripts. When dealing with *package.json* file, *preinstall* is extracted as a feature value. Figure 4(b) shows the AST of the target package and how the APIs and strings that perform malicious behaviors are extracted from the AST. Figure 4(c) shows the feature vector that reflects malicious behavior of creating a process and accessing a domain.

E. Step 5: Malicious Package Detection

This step applies the trained model (obtained in Step 3 of the training phase) to the feature representation of the target NPM packages (obtained in Step 4) to classify whether a target package is malicious or not, and if so, which lines of code are malicious.

Continuing with the preceding example, Figure 4(d) presents the detection result, not only showing that the target package is malicious but also pinpointing the malicious lines of code which are highlighted in red and italic.

III. The MalnpmDB Dataset

Rather than proposing just another dataset for benchmarking malicious NPM package datasets, we propose the desired characteristics of such datasets to guide us in preparing the MalnpmDB dataset.

A. Desired Characteristics

We propose the following desired characteristics of benchmarking datasets for malicious NPM package detectors.

- **Size**. A dataset should contain as many examples as possible, including both malicious and benign examples (i.e., NPM packages).

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

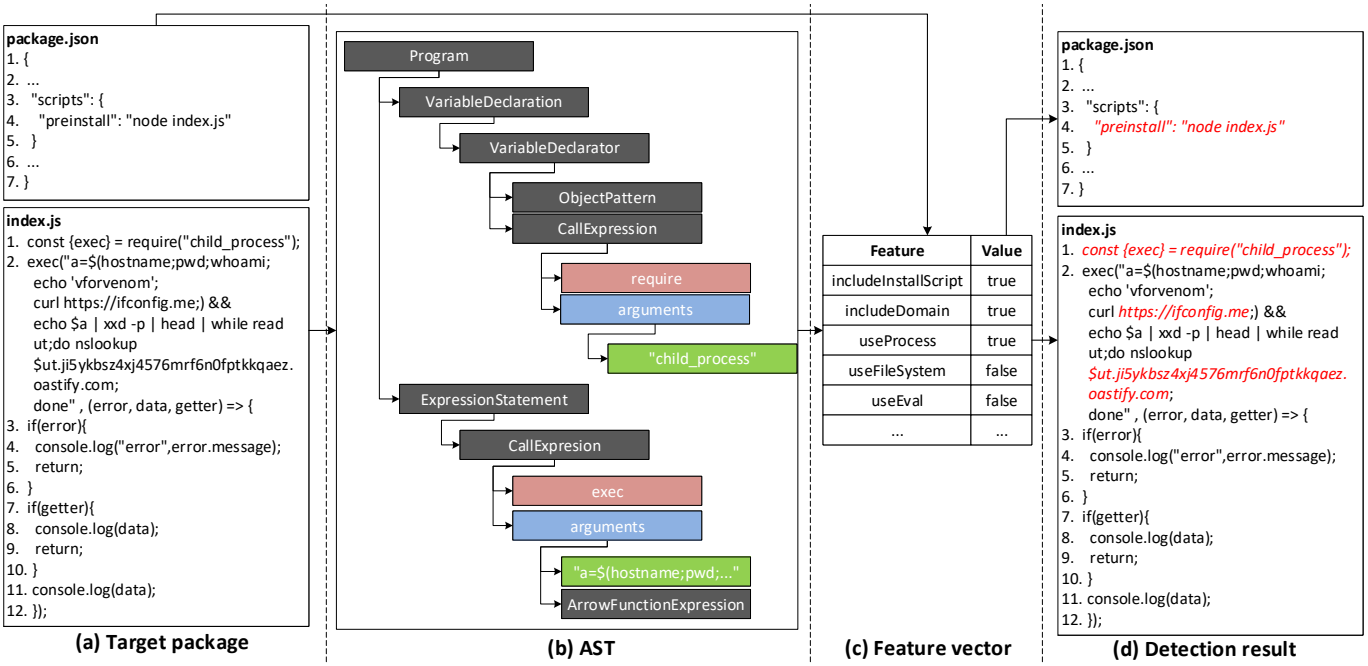

Figure 4: An example showing a target package is detected as malicious where malicious code is highlighted in red and italic.

- **Ground truth**. Each example is associated with its ground-truth label, where malicious examples (i.e., packages) are accompanied by the description of their malicious behaviors, which are not exhibited by the benign examples.
- **Duplication**. This refers to that a dataset should not contain any duplicate examples, or alternatively only contain a small fraction of duplicate examples. This is relevant because attackers often publish a malicious package with different package names, and thus these duplicate examples, which are different only in terms of package names, would affect the training and evaluation of detectors. Therefore, it is important to eliminate such duplicate examples when applicable.
- **Representativeness**. A dataset ideally should contain all kinds of malicious behaviors of NPM packages, and reflect the distribution of malicious behaviors in the real world so that the trained model can be well generalized. As an approximate measure of this metric, one may consider using the Vendi Score [19] to measure the diversity of malicious code. The Vendi Score measures the diversity of examples in a dataset and is thus considered suitable for our purpose. Moreover, we consider the distribution of the 7 common behaviors of NPM packages [20]: file system, network, process, operating system, code execution, obfuscation, and install script.
- **Balance**. A dataset should be balanced in terms of the ratio of malicious vs. benign examples and the ratio between the different kinds of malicious behaviors (i.e., ideally equal).

B. Constructing the MalnpmDB Dataset

Under the guidance of the desired characteristics mentioned above, we construct the MalnpmDB dataset as follows. First, we review the existing datasets for the same purpose [7], [12], [15] and the malicious NPM packages collected by the open-source community [21], [22] with respect to the characteristics mentioned above to identify their weaknesses. We find that the existing datasets have the following weaknesses: There are many duplicate examples in Knife [15] and a few duplicate examples in MalOSS [7]; the implication of these issues will be seen when we use these datasets to evaluate the effectiveness of detectors. Moreover, it is difficult to obtain the complete source code of malicious packages in Snyk [21]. These weaknesses suggest that we should propose a higher-quality dataset by collecting more malicious examples with source code while eliminating duplicates.

Table I: Sources of malicious NPM packages we collected

Source	Knife [15]	MalOSS [7]	Dockers	Enterprises	Total
#packages	2,086	567	1,239	2,897	6,789

Second, under the guidance of the desired characteristics and the findings about the weaknesses of existing datasets, we collect malicious NPM packages by searching projects on GitHub as well as old versions of Docker and mirrors based on the list of malicious NPM packages we identified. We also communicate with security field enterprises to obtain malicious packages they encountered in the past. This allows us to collect 6,789 malicious packages from the various sources described in Table I.

Third, we clean up the malicious examples by removing duplicate packages as attackers indeed reuse malicious packages under different package names. Two packages are duplicates of each other as long as they have (i) the same *scripts* field in their *package.json* files and (ii) the same code files. Note that (i) can be justified by the fact that the *scripts* field in the *package.json* file contains shell commands that will be automatically executed but the other fields (e.g.,

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

Table II: Datasets comparison

Metrics	Knife [15]	MalOSS [7]	MalnpmDB
Size (# of malicious packages)	2,086	567	3,258
Duplication (%)	77.3	18.2	0.0
Vendi Score	3.42	2.94	7.61

package name, version, description, and author's information) are irrelevant to code execution; whereas, (ii) can be justified by the fact that only code files are relevant to whether a package is malicious or benign, but explanatory files (e.g., *README.md*, *LICENSE*, *package-lock.json*) are not.

Fourth, we manually check that the remaining packages are indeed malicious. Each malicious package is reviewed by two researchers to confirm its maliciousness and classify its malicious behavior. Only when the labels and the malicious behavior classifications given by the two researchers are consistent will the package be considered malicious; in the case of dispute among the two researchers, the package will be reviewed by a third researcher. This takes four weeks of three experienced researchers, who also refine the *malicious* label of an example with a specific type of maliciousness mentioned in Table III (e.g., file system and network). This leads to 3,258 verified malicious NPM packages.

Fifth, we identify benign packages by treating the most popular packages as benign because they have likely been scrutinized by the community. To balance the malicious and benign examples in our dataset and consider the number of malicious examples after cleaning, we download the top 5,000 popular NPM packages [23]. Since some of these packages cannot be downloaded successfully, we obtain the 4,051 packages as benign examples. We advocate using balanced datasets for two reasons: (i) this aligns with conventional dataset construction practices; and (ii) it enables the model to learn more comprehensive malicious behavior patterns while minimizing false negatives. That is, the dataset contains 3,258 malicious NPM packages and 4,051 benign packages, leading to a total of 7,309 packages.

C. Analyzing the MalnpmDB Dataset

Now we show that the MalnpmDB dataset possesses the desired characteristics mentioned above. First, in terms of size, Table II shows that MalnpmDB contains a large number of malicious examples. Note that we focus on the size of malicious examples because they are often more difficult to obtain than benign examples.

Second, in terms of ground-truth, we manually verify that the malicious examples are indeed malicious, while using heuristics to identify benign examples—deeming the most popular packages as benign as they have been scrutinized by the community.

Third, in terms of duplication, Table II shows that MalnpmDB has 0% duplication in malicious packages, but the existing datasets, Knife [15] and MalOSS [7], have a significant percentage of duplications in malicious packages. It is interesting to note that after removing duplicate packages, Knife and MalOSS have about the same small number of malicious packages. In the following experiments (RQ2), we test the effect of the same model on different data sets. Too many repeated examples will cause the model to overfit.

Fourth, in terms of representativeness via the Vendi Score, our dataset exhibits a higher Vendi Score and thus higher representativeness. In terms of the distribution of malicious behaviors, we extract feature vectors from the malicious examples and then perform dimensionality reduction via the t-SNE algorithm [24] and perform clustering via the k-means algorithm [25]. Figures 5(a)-(c) present the scatter plots of feature vectors extracted from the three datasets, respectively. We observe that the distribution of feature vectors in Figure 5(c) is more scattered, indicating that the malicious behaviors in MalnpmDB (mal) (i.e., malicious examples in MalnpmDB) are more diverse and representative. By contrast, Knife and MalOSS are sparser and their malicious behaviors are monotonous: the feature vector distribution of Knife is similar to that of MalOSS despite that the former contains more data points (because the former contains duplicates).

Fifth, in terms of balance, MalnpmDB has a 44:56 ratio of malicious:benign packages. For optimal results, it is advisable to ensure a roughly equal number of examples for each class [26].

Insight 1: MalnpmDB is a better benchmark dataset than Knife and MalOSS for evaluating malicious NPM *packages detectors.*

IV. Experiments and Results

A. Experiments

We run experiments (the 5 steps of MalPacDetector) on a computer with an Intel(R) Xeon(R) Gold 6240 CPU running at 2.60GHz with 16GB memory storage. The LLM used in our experiment is GPT-3.5. The other tools used in the experiments include keyword extractor YAKE! and JavaScript compiler *babel*.

Selection of classifier to instantiate MalPacDetector in our experiments. As mentioned above, the NPM detector can be instantiated with many kinds of machine learning models, and the choice of a specific model depends on factors such as the size of the dataset. Although MalnpmDB has more malicious packages than any existing dataset, the number of malicious examples is still small. Thus, we choose the Random Forest (RF) model in our experiments because it can avoid overfitting, reduce variance, have a good generalization ability, evaluate the importance of features, and discover the interactions between features, which is beneficial for the analysis of our results. When training an RF model, we use a robust stratified κ-fold cross-validation to train and test a classifier, by partitioning the dataset into κ-1 subsets for training and using the remaining subset for testing.

Effectiveness metrics. To evaluate the effectiveness of a detector, we use the widely-used metrics: *accuracy*, *precision*, *False Negative Rate* (FNR), *False Positive Rate* (FPR), and *F1-measure* (F1). Accuracy is the overall correctness of a detector, namely the ratio of correctly predicted packages (including true positives and true negatives) to the total number of packages in question. Precision is the ratio of correctly predicted positive packages to the detected positive packages.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

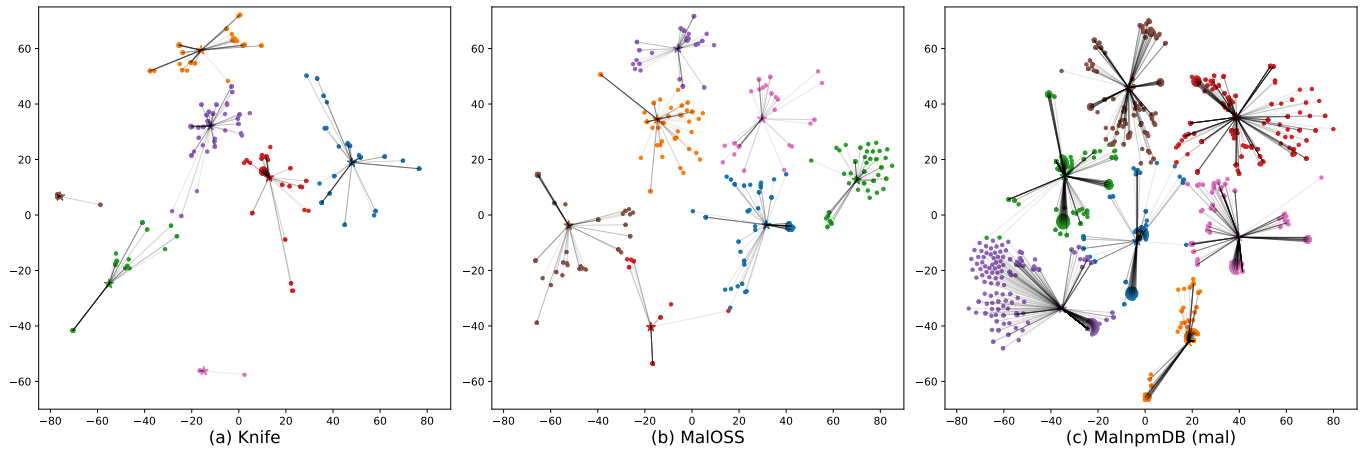

Figure 5: Scatter plots of feature vectors from (a) Knife, (b) MalOSS, (c) MalnpmDB (mal), where colors represent different types of malicious behaviors, and a straight line links from an example to the center of the cluster to which it belongs.

FNR is the ratio of false negative packages to the positive packages. FPR is the ratio of incorrectly predicted positive packages to negative packages. The overall effectiveness F1-measure is defined as $F1 = \frac{2 \cdot Precision \cdot (1\text{-}FNR)}{Precision + (1\text{-}FNR)}$.

Research Questions (RQs). Our experiments are geared towards answering:

- **RQ1**: How effective is LLM in generating features?
- **RQ2**: How effective is MalPacDetector in detecting malicious NPM packages with known ground truth?
- **RQ3**: How useful is MalPacDetector in detecting malicious NPM packages in the real world where the ground truth is unknown?

B. How Effective Is LLM? (RQ1)

Table III summarizes the 22 features extracted from MalnpmDB, with the assistance of GPT 3.5 as described above. We divide these 22 features into the following 7 categories.

- **File system**. Malicious JavaScript code can leverage the file system to conduct malicious activities, such as (i) stealing, tampering, and deleting sensitive data (e.g., login credentials) and personal documents and (ii) creating files or registry entries to persist on a victim's computer. This category has 3 features.
- **Network**. Malicious code may exfiltrate sensitive data from victim computers, causing network activities that may be leveraged to detect them. This category has 5 features.
- **Process**. Process-related malicious activities include downloading a malicious program from an attacker and executing it, stealing process environment variables, and running a backdoor program to receive and execute malicious instructions from the attacker. This category has 4 features.
- **Operating system**. Attackers can obtain information about the operating system running on a computer, its hostname, and its domain name. This information may help an attacker choose a specific attack or exploit. An attacker may execute some system calls or commands to perform malicious actions, such as creating, deleting, or modifying files and starting external processes. This category has one feature.
- **Code execution**. Malicious code often leverages dynamic code execution to perform actions, such as installing malware, exfiltrating sensitive data, and evading detection. These are often conducted by leveraging the *eval* function, leading to one feature.
- **Obfuscation**. Malicious code often employs obfuscation techniques to bypass security controls and evade the detection of security tools. Common obfuscation methods include encoding strings into the base64 or byte format, using the AES encryption algorithm, and using a compression tool. This category has 7 features.
- **Install script**. In each NPM package, there is a *package.json* file that contains metadata about the package. This allows package writers to customize their script commands, including *preinstall*, *install*, and *postinstall*, which are automatically executed before, during, and after package installation, respectively. Attackers can use this mechanism to automatically execute malicious instructions before, during, and after installation. This leads to the feature that considers the JavaScript code files that are involved in the install command as part of the install script.

Discovery of new malicious behavior feature. Table III compares the features we identified by the LLM with the features summarized in other papers [8], [11], [12], [27]. We can obtain all the malicious behavior features summarized in other papers through the LLM. In addition, we also found two new malicious behavior features. These features are related to a novel obfuscation method that achieves malicious behavior by inserting base64-obfuscated code into script files. By applying the feature set with two new features to the entire detection tool, we achieve a significant reduction in the false positive rate on the MalnpmDB dataset—from 4.6% down to 1.3%.

Performance comparison between different LLMs. We test LLMs, such as GPT-4 and Deepseek-R1, against the malicious samples in MalnpmDB. GPT-3.5 detects 8,793 malicious be-

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

Table III: Features generated with the assistance of GPT-3.5, from MalnpmDB

Feature category	Feature name	Feature description	Others	MalnpmDB
File system	useFileSystem	Use file system related libraries or functions in code	✓	✓
	useFileSystemInScript	Use file system related libraries or functions in the install script	✓	✓
	includeSensitiveFiles	Include sensitive files in string	✓	✓
Network	includeIP	Include IP	✓	✓
	includeDomain	Include domain name	✓	✓
	includeDomainInScript	Include domain name in the install script	✓	✓
	useNetwork	Use network related libraries or functions in code	✓	✓
	useNetworkInScript	Use network related libraries or functions in the install script	✓	✓
Process	useProcess	Use process related libraries or functions in code	✓	✓
	useProcessInScript	Use process related libraries or functions in the install script	✓	✓
	useProcessEnv	Use process environment related libraries or functions in code	✓	✓
	useProcessEnvInScript	Use process environment related libraries in the install script	✓	✓
Operating system	useOperatingSystem	Use operating system related libraries or functions in code	✓	✓
Code execution	useEval	Call *eval* function in code	✓	✓
Obfuscation	useBase64Conversion	Use base64 conversion in code	✓	✓
	useBase64ConversionScript	Use base64 conversion in the install script	-	✓
	includeBase64String	Include base64 string in code	✓	✓
	includeBase64StringInScript	Include base64 string in the install script	-	✓
	includeByteString	Include byte string in code	✓	✓
	useBuffer	Use *Buffer* library or functions in code	✓	✓
	useEncrytAndEncode	Use encryption and encoding libraries or functions in code	✓	✓
Install script	includeInstallScript	Have at least one install script in *package.json*	✓	✓

haviors (e.g., Figure 3(b)) but GPT-4 detects 126 more, and Deepseek-R1 detects 74 more. However, most of these newly detected malicious behaviors are similar to the known ones and did not result in the generation of new features. This partly owes to the feature threshold mentioned in Algorithm 1, which renders previously unseen malicious behaviors not to be summarized as features (i.e., limited capabilities in dealing with new behaviors). Another reason is that the number of malicious samples in MalnpmDB is limited, and the improvement of the performance of LLM is difficult to bring significant results.

Insight 2: LLM can identify malicious behaviors from NPM *packages and summarize them into features.*

C. How effective is MalPacDetector? (RQ2)

We construct a feature extractor by utilizing the 22 features summarized in Table III and the malicious code snippets generated by GPT-3.5. This extractor serves as the foundation for our subsequent experimental evaluations.

Experiments of MalPacDetector with different datasets. To measure the effectiveness of MalPacDetector in detecting malicious NPM packages with known ground truth, we conduct experiments on 4 datasets: Knife [15] and its variant dubbed Knife* which is obtained by removing the duplicated examples (i.e., only one copy is kept), MalOSS [7], and our dataset MalnpmDB. Since Knife, thus Knife*, and MalOSS only contain malicious NPM packages, we use the same 4,051 benign packages in our MalnpmDB for their benign examples. We use a 4-fold cross-validation, meaning $\kappa = 4$ in these experiments. The time complexity of the experiments is insignificant, e.g., the training time of MalPacDetector with respect to MalnpmDB is 1,877 seconds and the average detection time for a package is 0.25 seconds (thus, we will omit to mention the time complexity in subsequent experiments).

Table IV summarizes the effectiveness of MalPacDetector when applied to 4 datasets. We observe that it performs poorly when applied to the MalOSS dataset, with a high false negative rate (26.9%). This can be attributed to the small number of malicious examples. When applied to the Knife dataset, it achieves the smallest FNR (2.8%) and FPR (0.1%). This can be attributed to the fact that the dataset contains many duplicate examples because after removing the duplicates (Knife*), the FNR increases substantially (to 26.4%) and the FNR increases to 1.6%.

Table IV: Effectiveness of MalPacDetector (metrics unit: %)

Dataset	Accuracy	Precision	FNR	FPR	F1
Knife [15]	98.8	99.3	2.8	0.1	98.2
Knife*	94.3	84.2	26.4	1.6	78.4
MalOSS [7]	95.4	88.6	26.9	1.4	80.1
MalnpmDB (our dataset)	95.8	98.0	7.5	1.3	95.2

Comparing MalPacDetector with other detectors. Since MalnpmDB is better than the other datasets, we use it to compare the effectiveness of MalPacDetector with that of the following *publicly available* state-of-the-art malicious NPM package detectors: (i) the OSS Detect Backdoor (ODB) [8], which detects malicious behavior in NPM packages through regular expression matching; (ii) MalWuKong [27], which proposes an innovative approach that integrates source code slicing, inter-procedural analysis, and cross-file inter-procedural analysis; (iii) Ohm et al.'s detector [11], which is instantiated as an RF model and detects malicious packages using manually defined features; (iv) the AMALFI detector [12], which is instantiated as a Support Vector Machine (SVM) model and considers the difference when the malicious package is updated. Note that ODB and MalWuKong do not require model training, so we directly test their effectiveness on the entire MalnpmDB dataset. Note that we do not consider the DONAPI detector [28] because its source code and dataset are not publicly available, and we do not consider the MalOSS detector [7] because it uses dynamic features that are specific to the MalOSS dataset but do not apply to all NPM packages

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

Table V: Comparison of detectors via MalnpmDB (columns 2-5 metrics unit: %)

Detector	Accuracy	Precision	FNR	FPR	F1	P-Value
ODB [8]	47.2	45.4	9.9	75.5	60.4	0.01
MalWuKong [27]	93.1	95.0	10.5	3.7	92.2	0.43
Ohm et al. [11]	93.5	97.9	12.5	**1.3**	92.4	0.54
AMALFI [12]	90.1	92.5	14.9	5.4	88.6	0.05
MalPacDetector	**95.8**	**98.0**	**7.5**	**1.3**	**95.2**	1.00

in the MalnpmDB. In these experiments, we also set $\kappa = 4$ (i.e., 4-fold cross-validation).

Table V summarizes the experimental results. We observe that Ohm et al.'s detector [11] with RF achieves the best effectiveness, with an F1 of 92.4%. ODB has an extremely high FPR (75.5%) and high FNR 9.9%, likely owing to its use of general rules. Nevertheless, MalPacDetector outperforms the other detectors in all metrics, with a precision of 98.0%, a FPR of 1.3%, and an F1 of 95.2%. This can be attributed to that the features generated by LLM capture the distinctions between malicious and benign packages.

To rigorously evaluate the performance differences among the models, we conduct Friedman and Nemenyi statistical tests [29]. Since MalnpmDB encompasses all available malicious samples in our collection and no additional malware datasets exist for testing, we enhance the conventional Friedman test methodology. Our improved approach employs 4-fold cross-validation to partition the dataset, creating pseudo-repeated observations that simulate multiple dataset scenarios. We record each classifier's performance metrics during every validation fold, using F1-scores as our statistical measure.

The Friedman test yields a significant P-Value of 0.019, indicating statistically distinguishable performance among the classifiers. Subsequent Nemenyi post-hoc testing reveals specific differences between classifiers, as detailed in the "P-Value" column of Table V comparing other detectors against MalPacDetector. While Ohm et al.'s detector [11] and MalWuKong demonstrate comparable performance to our approach, their overall effectiveness remains inferior. The other two tools show substantially larger performance gaps compared to our solution.

Insight 3: MalPacDetctor outperforms the state-of-the-art malicious NPM *package detectors.*

Is RF the only option for instantiating MalPacDetector? To answer this question, we use MalnpmDB to train and compare 4 classifiers: Naive Bayes (NB), Multi-Layer Perceptron (MLP), RF, and SVM; we consider these models because they all offer good interpretability. We perform 4-fold cross-validation using different models and hyperparameters, and select the best hyperparameter based on the average F1 score.

Table VI shows the effectiveness of MalPacDetector when instantiated with the 4 classifiers, respectively. We observe that all instantiations have good precision. Nevertheless, MLP, RF, and SVM achieve better accuracy and FNR. In particular, RF excels by achieving the lowest FPR.

Insight 4: Random forest model is a better option for instantiating MalPacDetector, but other classifiers are also effective.

Table VI: Effectiveness of MalPacDetector with four machine learning classification models on our MalnpmDB dataset employing a 4-fold cross-validation methodology (metrics unit: %)

Model	Accuracy	Precision	FNR	FPR	F1
NB	93.4	97.3	12.5	2.0	92.1
MLP	95.7	97.0	6.8	2.4	95.0
RF	95.8	98.0	7.5	1.3	95.2
SVM	95.9	98.1	7.5	1.4	95.2

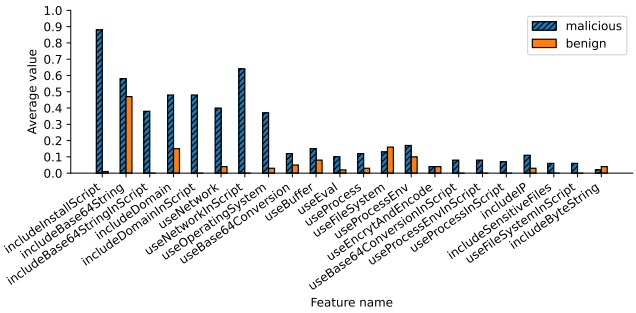

Figure 6: Barplot of average feature values, where malicious examples (i.e., packages) and benign ones exhibit a clear difference in the install script, meaning that attackers tend to insert malicious code into the install script.

Characterizing the features generated by LLM. As described in Step 1 of MalPacDetector, the feature generation process relies on a combination of domain expertise and statistical analysis. Moreover, certain features are customized to JavaScript packages. As highlighted in Figure 6, the features corresponding to the install script are *includeInstallScript*, *includeBase64StringInScript*, *includeDomainInScript*, *useNetworkInScript*, *useBase64ConversionInScript*, *useProcessEnvInScript*, *useProcessInScript*, and *useFileSystemInScript*, which have an average value close to 0 for benign examples and thus show a significant discrepancy between the benign examples and the malicious examples. This can be attributed to the fact that installation commands such as *preinstall* are often used to prepare the environment for installation (e.g., installing required dependencies, creating specific file structures, or setting environment variables). In most cases, *preinstall* requires only common shell commands and does not require JavaScript code to be executed; even when code execution is required, access to the libraries that are associated with these features is not necessary. Note that commands *install* and *postinstall* are similar. The values of features in code such as *includeDomain*, *useNetwork*, *useOperatingSystem*, *useEval*, *useProcess*, and *includeIP* show obvious discrepancies between malicious and benign examples. Other features, such as *includeBase64String*, *useFileSystem*, *useBuffer*, and *useProcessEnv*, have a similar average value for benign and malicious examples, which can be attributed to the fact that all examples often use these basic and common modules. The preceding discussion underscores the effectiveness of the features generated by LLM in discriminating between benign and malicious packages.

Insight 5: LLM, or more specifically GPT 3.5, can effectively

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

generate features to distinguish benign and malicious NPM *packages.*

D. How Useful Is MalPacDetector? (RQ3)

To evaluate the usefulness of MalPacDetector in practice where the ground truth is unknown, we apply MalPacDetector to the newly released packages on the NPM registry over three weeks. Due to the balanced nature of our training dataset, direct deployment of our detection model in real-world environments may result in an unacceptably high false positive. To address this issue, we adopt the methodology proposed by AMALFI [12], to implement a daily manual review process for all detected packages. Furthermore, we incorporate these verified packages (including both benign and malicious ones) into our training dataset, enabling iterative model refinement through continuous retraining.

We analyze 269,175 packages using MalPacDetector, which initially identifies 1,604 potentially malicious packages (most of them occur in the first three days). Through manual verification, we confirm that 39 are indeed malicious packages. We report them to the NPM security team, which confirmed the maliciousness of all of them. As a result, these 39 packages have been removed from the NPM registry.

Table VII: The 39 malicious NPM packages detected by MalPacDetector and their attack methods, where "A1" means Shell in installation, "A2" means Shell in code, "A3" means Pure malicious code, "A4" means DNS resolution, "A5" means Obfuscation, "✓" means that a package contains an attack method, and "-" means that a package does not contain an attack method.

Package name	#Downloads	A1	A2	A3	A4	A5
emails-helper-2.0.20	539	-	-	✓	✓	-
emails-helpers-2.0.20	0	-	-	✓	✓	-
hydra-consent-app-express-2.0.0	66	-	-	✓	-	-
rust-functions-1.0.2	73	-	-	✓	-	-
web3-provider-patchers-1.0.2	0	-	-	-	-	✓
wallet-watch-asset-1.0.2, 21.0.2	213	-	-	-	-	✓
wallet-add-chain-1.0.2	47	-	-	-	-	✓
metronome-ui-1.0.2, 21.0.2	186	-	-	-	-	✓
chain-list-2.0.0	106	-	-	-	-	✓
master-oracle-lib-2.0.0	60	-	-	-	-	✓
vesper-synth-user-lib-2.0.0	64	-	-	-	-	✓
metronome-synth-user-lib-1.0.2	0	-	-	-	-	✓
betterbit-frame-pkg-2.0.0	58	-	-	-	-	✓
dm_commons_utilities-99.99.0	0	-	-	✓	-	-
hardhat-gas-report-1.1.18	245	✓	-	-	-	-
helio_tawa-5.0.1	196	-	-	✓	-	-
@gds-web-ui/core-1.0.0	0	-	-	✓	-	-
@gds-web-ui/sodalite-0.23.1	0	-	-	✓	-	-
@grabdefence/trust-feature-1.0.5-rc.1	0	-	-	-	✓	-
paysecure-9.0.3	0	-	-	✓	-	-
paysecuretest-99.9.9	0	-	-	✓	-	-
grablink-web-sdk-1.1.2	0	-	-	✓	-	-
kwaishop-radar-99.9.1	0	-	-	-	-	✓
vforvenom-1.999.0	0	-	✓	✓	-	-
pb-styles-v1-5.1.1	0	-	-	✓	-	-
zara-mkt-core-1.0.0, 9.9.1	74	✓	-	-	-	-
npm-random-gen-1.0.0, 1.0.1	0	✓	-	-	-	-
send-orchestrator-event-lambda-8.8.8	77	-	-	✓	-	-
walletconnect-website-4.4.4	0	-	-	✓	-	-
subspace-relayer-front-end-4.4.4	0	-	-	✓	-	-
pathkit-local-9.9.9	60	✓	-	-	-	-
producer-journey-1.0.4	378	-	-	✓	-	-
adidas-data-mesh-9.9.8	760	✓	-	-	-	-
inteken-app-client-9.9.1	303	✓	-	-	-	-
surf-sharekit-frontend-9.9.7	206	-	-	✓	-	-
symbl_ai-1.0.0	66	-	-	✓	-	-
puppeteer-example-0.1.10	1,187	-	-	✓	-	-
mfp-food-diary-0.1.2	140	✓	-	-	-	-
kendo-react-messages-1.999.0, 1.999.1	0	-	-	✓	-	-

Table VII summarizes the 39 malicious packages detected by MalPacDetector. We observe that 22 packages have been downloaded multiple times, but the others have not been downloaded because we inform the NPM security team about their maliciousness rapidly. We find that the 39 packages are mainly about stealing user or host computer information. We summarize their attack and anti-detection methods in 5 categories:

- **Shell scripts in installation**. This type of package uses shell scripts to leverage *preinstall*, *install*, or *postinstall* to wage attacks before, during, or after the installation process that is described in *scripts* field of the *package.json* file.
- **Shell scripts in code.** This type of malicious package uses shell scripts in JavaScript code to attack the user's host.
- **Pure malicious code**. The payload steals users' information (e.g., home path, hostname, username, and DNS servers' information) and sends it to the remote server. It does not employ obfuscation or other techniques to evade detection.
- **DNS resolution**. In this case, the stolen information is sent through the DNS resolution protocol rather than the common communication protocols such as HTTP or HTTPS. It splits data into many pieces, encodes and wraps a piece of data into the start of the domain name, and then adds some random bytes or salt into the piece to avoid DNS caching.
- **Obfuscation.** This attack uses obfuscation to lower readability, such as splitting keywords into several parts and storing them in a "keyword dictionary". When calling functions, it uses a dictionary lookup function to search for keywords via their index. It also contains much junk code (e.g., adding useless conditional branch statements and wrapping a normal function call statement into multiple function calls). It periodically executes a function that interrupts the debugger to prevent security personnel from dynamically debugging the code.

Table VII summarizes the attack methods of the 39 malicious packages detected by MalPacDetector. We observe that the two most common attacks are *pure malicious code* and *obfuscation*, which are used in 20 and 10 packages, respectively. The third most frequent attack is *shell scripts in installation* (7 packages).

Insight 6: MalPacDetector is useful in detecting malicious NPM *packages in the real world.*

V. RELATED WORK

We divide related prior studies into three categories: malicious NPM package detection, malicious NPM package datasets, and the application of LLM to code security.

A. Malicious NPM Package Detectors

There are two types of approaches to detecting malicious NPM packages: program analysis-based and machine learning-based detectors.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

Program analysis-based detectors. These detectors use rules to detect malicious code while leveraging other information (e.g., historical versions, developers, and software components). These detectors often suffer from a high false positive rate or false negative rate [8], [30]. Duan et al. [7] leveraged a combination of static analysis and dynamic analysis, together with package metadata, to build an analysis pipeline and detect malicious packages. However, this detector requires the manual definition and updating of detection rules. Scalco et al. [14], [31] examined the differences between the package and its purported source code to detect malware. However, code difference is a weak indicator of attack; thus, the detector can miss many attacks. Huang et al. [28] combined code reconstruction techniques and static analysis, and extracted API call sequences to confirm and detect obfuscated content. However, the effectiveness of their static analysis part still depends on the selection of features. Detectors such as Microsoft's Application Inspector [30] and OSS Detect Backdoor [8] provide regular expression-based scanning with a high false positive rate.

Machine learning-based detectors. These detectors are trained via supervised learning from manually-defined features (e.g., [11], [12], [28]) or unsupervised learning but still from manually defined features (e.g., [10], [13], [32]–[34]). Ohm et al. [11] analyzed three commonly employed supervised machine learning techniques and concluded that pre-selecting a number of packages for further manual analysis could help detectors work. Sejfia and Schafer [12] proposed detectors consisting of three complementary techniques in the context of NPM packages, by leveraging manually-defined features. By contrast, we use LLM to learn features automatically. Experimental results show that our detector simultaneously achieves a lower false positive rate and a lower false negative rate. In this study, we propose and investigate how to apply LLM to define/generate effective features.

B. Malicious NPM Package Datasets

At present, the mainstream way to obtain datasets is mainly through communities and academic research. NPM official regularly removes malicious packages, which makes it extremely difficult for us to obtain source code datasets.

Communities. In the public community, only malicious package names and version information can be obtained, and it is difficult to obtain complete malicious packages with source code. In open-source communities such as GitHub [22], researchers collect lists of malicious packages and classify them. Public detection platforms such as Snyk [21] regularly publish information about malicious packages they discovered. In comparison, we collect a list of malicious packages in the past three years and the source code of the packages from some mirror sources and Docker [34].

Academic research. The dataset in academic research is another important source, but currently, there are fewer works on malicious package detection and the dataset used is also single. The Backstabber's Knife Collection [15] is a widely used dataset, and it takes the lead in collecting and analyzing malicious packages. However, this dataset has monotonous malicious behaviors and many repetitive segments. MalOSS [7] utilizes recent malicious packages gathered from the community to create experimental data; however, the limited amount of dataset is insufficient to meet the demands of machine learning. AMALFI [12] obtained some malicious examples from the NPM official and built a dataset, but the author only disclosed the version information of the malicious packages. Yu et al. [35] used an LLM to convert other types of malicious code into JavaScript code. This solution can enrich the types of malicious behaviors, but there is a gap between the converted malicious code and the malicious code in the real world. The above three datasets cannot meet the needs of our model in terms of quantity and quality, so we built our own dataset.

C. Application of LLM to Code Security

LLM has been applied for code detection and test generation purposes [36]–[38]. Closely related to our studies are [39], which applies LLM to detect malicious NPM and Python packages, and [40], which applies LLM to detect small NPM packages. The present study is the first to apply LLM to generate features from malicious packages and then use these features to train models that are faster and/or more lightweight than deep learning models, which are also not feasible in our setting because of the limited dataset size.

VI. DISCUSSIONS

A. Limitations

The present study has several limitations. First, the main challenge in malicious NPM package detection lies in anti-detection technology, especially code obfuscation. While AST allows us to handle basic code minification to some degree, it is ineffective against advanced obfuscation techniques, such as complex code compression. In these cases, we cannot accurately extract original feature vectors but must process such samples uniformly as obfuscated code. Considering how attackers inject malicious code into normal packages, we can prevent obfuscation to some extent. Further research needs to be conducted to deal with code obfuscation attacks.

Second, MalPacDetector uses static detection technology, which means that it has limited capability in dealing with dynamic behaviors. For instance, when dealing with unknown domain names and scripts, manual review is necessary. A package has multiple malicious features but does not perform malicious behavior. This type of package is rare in our dataset, but it is foreseeable that the model will have a high false positive rate for such packages.

Third, the quality of feature generation heavily depends on the dataset. When we adapt this approach to Python packages, the results are significantly less impressive than their counterparts with respect to NPM packages. This can be attributed to the limited quantity and low diversity of malicious samples available in Python.

Fourth, the feature generation relies on the performance of the LLM in question. For new types of malicious packages,

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

the LLM may not be able to extract the malicious behavior snippets, leading to ineffective features. This issue needs to be address by more studies.

B. Threats to Validity

In our approach of using LLMs to generate features, we utilize all malicious samples from MalnpmDB. This means the training and test sets divided from MalnpmDB in subsequent experiments cannot be guaranteed as completely independent.

This compromise stems from two practical constraints: (i) A fixed feature set and feature extractor are essential. Given the limited number of malicious samples and the need to prevent overfitting, 4-fold cross-validation proves critical specifically, only the training-test set partitioning varies when computing the model's average performance. Constructing separate feature extractors for each fold would compromise experimental rigor. (ii) The feature set and extractor are constructed based on the observed distribution of malicious behaviors. Under ideal conditions, random dataset partitioning maintains this behavioral distribution, ensuring the test set involved in feature generation does not affect subsequent model training.

To partially demonstrate our tool's capability in detecting completely unknown samples, we provide real-world detection results. Conducting more rigorous quantitative evaluation remains a direction for our future work.

VII. Conclusion

We have presented MalPacDetector, a malicious NPM package detector, and showed it is effective and useful. One key innovation is to use LLM to guide the generation of features to describe the malicious behaviors of NPM packages. We have not only constructed a new dataset, MalnpmDB, which advancaes the state of the art, but also proposed a set of desired characteristics that should be satisfied by a good benchmark dataset. The limitations of MalPacDetector serve as interesting open problems for future research.

References

[1] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. S. Maddila, and L. A. Williams, "What are weak links in the npm supply chain?" in *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, Pittsburgh, PA, USA*, 2022, pp. 331–340.

[2] A. Bagmar, J. Wedgwood, D. Levin, and J. Purtilo, "I know what you imported last summer: A study of security threats in thepython ecosystem," *arXiv preprint arXiv:2102.06301*, 2021.

[3] I. Koishybayev and A. Kapravelos, "Mininode: Reducing the attack surface of node.js applications," in *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID), San Sebastian, Spain*, 2020, pp. 121–134.

[4] X. Jiang, L. Meng, S. Li, and D. Wu, "Active poisoning: efficient backdoor attacks on transfer learning-based brain-computer interfaces," *Science China Information Sciences*, vol. 66, no. 8, 2023.

[5] Eslint-scope, https://github.com/advisories/GHSA-hxxf-q3w9-4xgw.

[6] M. Zimmermann, C. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *Proceedings of the 28th USENIX Security Symposium, Santa Clara, CA, USA*, 2019, pp. 995–1010.

[7] R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, "Towards measuring supply chain attacks on package managers for interpreted languages," in *Proceedings of the 28th Annual Network and Distributed System Security Symposium (NDSS), Virtual Event*. The Internet Society, 2021.

[8] Microsoft OSS Detect Backdoor System, https://github.com/microsoft/OSSGadget/wiki/OSS-Detect-Backdoor.

[9] D. L. Vu, I. Pashchenko, F. Massacci, H. Plate, and A. Sabetta, "Towards using source code repositories to identify software supply chain attacks," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA*, 2020, pp. 2093–2095.

[10] M. Ohm, A. Sykosch, and M. Meier, "Towards detection of software supply chain attacks by forensic artifacts," in *Proceedings of the 15th International Conference on Availability, Reliability and Security, Virtual Event, Ireland*, 2020, pp. 65:1–65:6.

[11] M. Ohm, F. Boes, C. Bungartz, and M. Meier, "On the feasibility of supervised machine learning for the detection of malicious software packages," in *Proceedings of the 17th International Conference on Availability, Reliability and Security, Vienna, Austria*, 2022, pp. 127:1–127:10.

[12] A. Sejfia and M. Schäfer, "Practical automated detection of malicious npm packages," in *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA*, 2022, pp. 1681–1692.

[13] M. Ohm, L. Kempf, F. Boes, and M. Meier, "Supporting the detection of software supply chain attacks through unsupervised signature generation," *arXiv preprint arXiv:2011.02235*, pp. 1–20, 2020.

[14] S. Scalco, R. Paramitha, D. L. Vu, and F. Massacci, "On the feasibility of detecting injections in malicious npm packages," in *Proceedings of the 17th International Conference on Availability, Reliability and Security, Vienna, Austria*, 2022, pp. 115:1–115:8.

[15] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, Lisbon, Portugal*, 2020, pp. 23–43.

[16] R. Campos, V. Mangaravite, A. Pasquali, A. Jorge, C. Nunes, and A. Jatowt, "Yake! keyword extraction from single documents using multiple local features," vol. 509, 2020, pp. 257–289.

[17] GPT-3.5, https://platform.openai.com/docs/models/gpt-3-5.

[18] Babel, https://babeljs.io/.

[19] D. Friedman and A. B. Dieng, "The Vendi Score: A diversity evaluation metric for machine learning," *Transactions on Machine Learning Research*, 2023.

[20] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "Sok: Taxonomy of attacks on open-source software supply chains," in *Proceedings of the 44th IEEE Symposium on Security and Privacy (S&P) , San Francisco, CA, USA*, 2023, pp. 1509–1526.

[21] Snyk Vulnerability Database of npm, https://security.snyk.io/vuln/npm.

[22] GitHub Advisory Database of npm, https://github.com/advisories?query=type%3Areviewed+ecosystem%3Anpm.

[23] Libraries.io of npm, https://libraries.io/npm.

[24] L. van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[25] A. M. Ikotun, A. E. Ezugwu, L. Abualigah, B. Abuhaija, and J. Heming, "K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data," *Information Sciences*, vol. 622, pp. 178–210, 2023.

[26] R. Mohammed, J. Rawashdeh, and M. Abdullah, "Machine learning with oversampling and undersampling techniques: Overview study and experimental results," in *Proceedings of the 11th International Conference on Information and Communication Systems (ICICS)*, 2020, pp. 243–248.

[27] N. Li, S. Wang, M. Feng, K. Wang, M. Wang, and H. Wang, "Malwukong: Towards fast, accurate, and multilingual detection of malicious code poisoning in oss supply chains, luxembourg," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 1993–2005.

[28] C. Huang, N. Wang, Z. Wang, S. Sun, L. Li, J. Chen, Q. Zhao, J. Han, Z. Yang, and L. Shi, "DONAPI: malicious NPM packages detector using behavior sequence knowledge mapping," in *Proceedings of the 33rd USENIX Security Symposium, USENIX Security , Philadelphia, PA, USA*, 2024.

[29] S. Garcia and F. Herrera, "An extension on" statistical comparisons of classifiers over multiple data sets" for all pairwise comparisons." *Journal of machine learning research*, vol. 9, no. 12, 2008.

[30] Microsoft OSS ApplicationInspector, https://github.com/microsoft/ApplicationInspector.

[31] D. L. Vu, F. Massacci, I. Pashchenko, H. Plate, and A. Sabetta, "Lastpymile: Identifying the discrepancy between sources and packages," in *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece*, 2021, pp. 780–792.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Information Forensics and Security. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIFS.2025.3580336

[32] K. A. Garrett, G. Ferreira, L. Jia, J. Sunshine, and C. Kästner, "Detecting suspicious package updates," in *Proceedings of the 41st International Conference on Software Engineering: New Ideas and Emerging Results, Montreal, QC, Canada*, 2019, pp. 13–16.
[33] M. Ohm, L. Kempf, F. Boes, and M. Meier, "Towards detection of malicious software packages through code reuse by malevolent actors." Gesellschaft für Informatik, Bonn, 2022.
[34] D. U. Brand, O. Stussi, and E. Wåreus, "Supply chain attacks in open source projects," Master's thesis, LUND University, 2022.
[35] Z. Yu, M. Wen, X. Guo, and H. Jin, "Maltracker: A fine-grained npm malware tracker copiloted by LLM-enhanced dataset," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1759–1771.
[36] W. Tang, M. Tang, M. Ban, Z. Zhao, and M. Feng, "CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection," *Journal of Systems and Software*, vol. 199, p. 111623, 2023.
[37] K. Zhang, D. Wang, J. Xia, W. Y. Wang, and L. Li, "ALGO: synthesizing algorithmic programs with generated oracle verifiers," in *Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS), New Orleans, LA, USA*, 2023.
[38] M. Alqarni and A. Azim, "Low level source code vulnerability detection using advanced BERT language model," in *Proceedings of the 35th Canadian Conference on Artificial Intelligence, Toronto, Ontario, Canada*, 2022, pp. 1–11.
[39] J. Zhang, K. Huang, B. Chen, C. Wang, Z. Tian, and X. Peng, "Malicious package detection in npm and pypi using a single model of malicious behavior sequence," *arXiv preprint arXiv:2309.02637*, 2023.
[40] N. Zahan, P. Burckhardt, M. Lysenko, F. Aboukhadijeh, and L. Williams, "Shifting the lens: Detecting malware in npm ecosystem with large language models," *arXiv preprint arXiv:2403.12196*, 2024.

Jian Wang received the bachelor's degree from Huazhong University of Science and Technology (HUST), Wuhan, China, in 2016. He is currently pursuing a Ph.D degree with the School of Computer Science and Technology, HUST, under the supervision of Deqing Zou. His research interests include malicious code detection.

Zhen Li (Member, IEEE) received the Ph.D. degree from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2019. She is currently an Associate Professor of Cyberspace Security at HUST. Her research interests mainly include software security and AI security.

Jixiang Qu received the bachelor's degree from Huazhong University of Science and Technology (HUST), Wuhan, China, in 2018. He is currently pursuing a master's degree with the School of Computer Science and Technology, HUST, under the supervision of Yunhe Zhang. His research interests include malicious code detection.

Deqing Zou received the PhD degree from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2004. He is currently a professor of Cyberspace Security at HUST. His main research interests include system security, trusted computing, virtualization, and cloud security. He has always served as a reviewer for several prestigious journals, such as IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Computers, IEEE Transactions on Parallel and Distributed Systems, and IEEE Transactions on Cloud Computing. He is on the editorial boards of four international journals and has served as PC chair/PC member of more than 40 international conferences.

Shouhuai Xu (Senior Member, IEEE) received the PhD degree in computer science from Fudan University, Shanghai, China, in 2000. He is the Gallogly Chair Professor in cybersecurity with the Department of Computer Science, University of Colorado Colorado Springs (UCCS). Prior to joining UCCS, he was with the University of Texas at San Antonio. He pioneered the Cybersecurity Dynamics approach as the foundation for the emerging science of cybersecurity, with three pillars: first-principle cybersecurity modeling and analysis (the x-axis); cybersecurity data analytics (the y-axis, to which the present paper belongs); and cybersecurity metrics (the z-axis). He co-initiated the International Conference on Science of Cyber Security (SciSec) and is serving as its Steering Committee chair. He is/was an associate editor of IEEE Transactions on Dependable and Secure Computing (IEEE TDSC), IEEE Transactions on Information Forensics and Security (IEEE TIFS), and IEEE Transactions on Network Science and Engineering (IEEE TNSE).

Ziteng Xu joined Ant Technology Group Co., Ltd, Hangzhou, China, in 2017. He is currently a researcher in computer software. His research interests include code security and malicious code detection.

Zhenwei Wang joined Ant Technology Group Co., Ltd, Hangzhou, China, in 2016. He is currently a researcher in computer security. His research interests include the detection and analysis of third-party malicious packages.

Hai Jin (Fellow, IEEE) received the Ph.D. degree in computer engineering from the Huazhong University of Science and Technology in 1994. He was with The University of Hong Kong from 1998 to 2000 and a Visiting Scholar with the University of Southern California from 1999 to 2000. He is currently the Chair Professor of Computer Science and Engineering at the Huazhong University of Science and Technology (HUST), China. He has coauthored more than 20 books and published over 900 research articles. His research interests include computer architecture, parallel and distributed computing, big data processing, data storage, and system security. He is a Fellow of IEEE, a Fellow of CCF, and a Life Fellow Member of ACM. In 1996, he was awarded the German Academic Exchange Service Fellowship to visit the Technical University of Chemnitz, Germany. He received the Excellent Youth Award from the National Science Foundation of China in 2001.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/